

# Factorial Processing Walkthrough

## PLC Compiler Pipeline — Step-by-Step Trace

The Token Trio

Aye Khin Khin Hpone | Applegate T. Tun Oo | Win Htut Naing

(Yolanda Lim) st125970 | st126690 | st126687

---

**Companion document.** This document is a supplementary walkthrough submitted alongside the main report *From Source to Execution: Design and Implementation of a Statically-Typed Programming Language with Type Inference* (st125970, st126690, st126687). It provides a concrete, end-to-end trace of the compiler pipeline applied to a single factorial program, complementing the formal descriptions and grammar given in the main report. Readers are expected to consult the main report for language design, grammar specification, system architecture, and the automated test suite; this document focuses solely on illustrating how a real program flows through each pipeline stage.

This document traces `factorial(5) = 120` through every stage of the PLC compiler pipeline: lexical analysis, parsing, type checking, and tree-walking interpretation.

## 1 Source Program

Recursion is not supported by the type checker; factorial is computed using an accumulator variable inside a `while` loop.

```
1 def factorial(n) {  
2     result = 1;  
3     i = n;  
4     while (i != 0) {  
5         result = result * i;  
6         i = i - 1;  
7     }  
8     return result;  
9 }  
10  
11 x = 5;  
12 f = factorial(x);  
13 print(f);
```

Listing 1: Iterative factorial program

## 2 Stage 1 — Lexical Analysis

The lexer (`components/lexica.py`) scans source text into tokens, each carrying a type from `TokenType`, a lexeme, and a source position. Table ?? lists the complete token stream (54 tokens plus EOF).

Table 1: Token stream for the factorial program.

| Token type  | Lexeme    | Notes               |
|-------------|-----------|---------------------|
| DEF         | def       | Keyword             |
| IDENTIFIER  | factorial | Function name       |
| LPAREN      | (         |                     |
| IDENTIFIER  | n         | Parameter name      |
| RPAREN      | )         |                     |
| LBRACE      | {         | Begin function body |
| IDENTIFIER  | result    |                     |
| ASSIGN      | =         |                     |
| INT_LITERAL | 1         | Integer literal     |
| SEMICOLON   | ;         |                     |
| IDENTIFIER  | i         |                     |
| ASSIGN      | =         |                     |
| IDENTIFIER  | n         |                     |
| SEMICOLON   | ;         |                     |
| WHILE       | while     | Keyword             |
| LPAREN      | (         |                     |
| IDENTIFIER  | i         |                     |
| BANG_EQUAL  | !=        | Not-equal operator  |
| INT_LITERAL | 0         | Integer literal     |
| RPAREN      | )         |                     |
| LBRACE      | {         | Begin loop body     |
| IDENTIFIER  | result    |                     |
| ASSIGN      | =         |                     |
| IDENTIFIER  | result    |                     |
| STAR        | *         | Integer multiply    |
| IDENTIFIER  | i         |                     |

| Token type  | Lexeme    | Notes                |
|-------------|-----------|----------------------|
| SEMICOLON   | ;         |                      |
| IDENTIFIER  | i         |                      |
| ASSIGN      | =         |                      |
| IDENTIFIER  | i         |                      |
| MINUS       | -         | Integer subtract     |
| INT_LITERAL | 1         | Integer literal      |
| SEMICOLON   | ;         |                      |
| RBRACE      | }         | End loop body        |
| RETURN      | return    | Keyword              |
| IDENTIFIER  | result    |                      |
| SEMICOLON   | ;         |                      |
| RBRACE      | }         | End function body    |
| IDENTIFIER  | x         |                      |
| ASSIGN      | =         |                      |
| INT_LITERAL | 5         | Integer literal      |
| SEMICOLON   | ;         |                      |
| IDENTIFIER  | f         |                      |
| ASSIGN      | =         |                      |
| IDENTIFIER  | factorial | Callee name          |
| LPAREN      | (         |                      |
| IDENTIFIER  | x         | Argument             |
| RPAREN      | )         |                      |
| SEMICOLON   | ;         |                      |
| IDENTIFIER  | print     | Built-in function    |
| LPAREN      | (         |                      |
| IDENTIFIER  | f         |                      |
| RPAREN      | )         |                      |
| SEMICOLON   | ;         |                      |
| EOF         | (end)     | Signals end of input |

**Lexer notes.** Keywords (def, while, return) are emitted as dedicated token types; all other names including factorial and print are emitted as IDENTIFIER. The two-character token != is recognised

by consuming ! and peeking ahead for =. No float literals or dot-suffixed operators appear in this program.

**Token string (parser input).** Each token carries both a *lexeme* (the raw source text) and a *token type* (the name the parser acts on). The flat sequence of token types delivered to the parser is shown below, grouped by source statement:

```
DEF IDENTIFIER LPAREN IDENTIFIER RPAREN LBRACE
  IDENTIFIER ASSIGN INT_LITERAL SEMICOLON
  IDENTIFIER ASSIGN IDENTIFIER SEMICOLON
  WHILE LPAREN IDENTIFIER BANG_EQUAL INT_LITERAL RPAREN LBRACE
    IDENTIFIER ASSIGN IDENTIFIER STAR IDENTIFIER SEMICOLON
    IDENTIFIER ASSIGN IDENTIFIER MINUS INT_LITERAL SEMICOLON
  RBRACE
  RETURN IDENTIFIER SEMICOLON
RBRACE
IDENTIFIER ASSIGN INT_LITERAL SEMICOLON
IDENTIFIER ASSIGN IDENTIFIER LPAREN IDENTIFIER RPAREN SEMICOLON
IDENTIFIER LPAREN IDENTIFIER RPAREN SEMICOLON EOF
```

### Mini if-else walkthrough: input → lexeme → token → parse tree

The factorial program itself does not contain an if statement, but the professor specifically asked for a clean if-else example showing the full system process on separate lines. The example below presents that chain explicitly.

#### Input source string

```
if ( x == 0 ) { print(x); } else { print(y); }
```

#### Lexeme sequence

```
if ( x == 0 ) { print ( x ) ; } else
{ print ( y ) ; }
```

#### Token sequence delivered to the parser

```
IF LPAREN IDENTIFIER EQUAL_EQUAL INT_LITERAL RPAREN
LBRACE IDENTIFIER LPAREN IDENTIFIER RPAREN SEMICOLON RBRACE
ELSE
LBRACE IDENTIFIER LPAREN IDENTIFIER RPAREN SEMICOLON RBRACE
```

#### Lexeme-to-token definition

```
'if' -> IF
```

```

'else' -> ELSE
'(' -> LPAREN
')' -> RPAREN
'{' -> LBRACE
'}' -> RBRACE
'==' -> EQUAL_EQUAL
'0' -> INT_LITERAL
'x','y','print' -> IDENTIFIER

```

## CFG rules containing those tokens

```

if_stmt -> IF LPAREN expr RPAREN block else_part
else_part -> ELSE block | epsilon
block -> LBRACE stmt_list RBRACE
expr -> comparison | additive
comparison -> additive EQUAL_EQUAL additive

```

## Parse-tree view

```

statement
+-- if_stmt
| +-- IF
| +-- LPAREN
| +-- expr
| | +-- comparison
| | +-- additive
| | | +-- factor
| | | +-- IDENTIFIER(x)
| | +-- EQUAL_EQUAL
| | +-- additive
| | +-- factor
| | +-- INT_LITERAL(0)
| +-- RPAREN
| +-- block
| | +-- LBRACE
| | +-- statement
| | | +-- expr_stmt
| | | +-- expr
| | | | +-- additive
| | | | +-- term
| | | | +-- factor
| | | | +-- function_call
| | | | +-- IDENTIFIER(print)
| | | | +-- LPAREN
| | | | +-- args
| | | | | +-- expr
| | | | | +-- additive
| | | | | +-- term
| | | | | +-- factor
| | | | | +-- IDENTIFIER(x)
| | | | +-- RPAREN
| | +-- SEMICOLON
| +-- RBRACE
+-- else_part
+-- ELSE
+-- block
+-- LBRACE

```

```

| +-- statement
| | +-- expr_stmt
| | +-- expr
| | | +-- additive
| | | +-- term
| | | +-- factor
| | | +-- function_call
| | | +-- IDENTIFIER(print)
| | | +-- LPAREN
| | | +-- args
| | | | +-- expr
| | | | +-- additive
| | | | +-- term
| | | | +-- factor
| | | | +-- IDENTIFIER(y)
| | | +-- RPAREN
| | +-- SEMICOLON
| +-- RBRACE

```

This mini example makes the separation explicit: the *input string* contains raw characters, the *lexeme sequence* is the character grouping recognised by the lexer, the *token sequence* is the abstract vocabulary consumed by the parser, and the *parse tree* is the hierarchical concrete-syntax view produced after parsing that token sequence.

### 3 Stage 2 — Parsing

The recursive-descent parser consumes the token string and builds an Abstract Syntax Tree (AST). Assignments are distinguished from expression statements by one-token lookahead (IDENTIFIER followed by ASSIGN). Operator precedence is encoded structurally across three rule levels:

| Rule                     | Handles                                                      |
|--------------------------|--------------------------------------------------------------|
| <code>_expr()</code>     | Comparison (EQUAL_EQUAL, BANG_EQUAL)                         |
| <code>_additive()</code> | Addition / subtraction (PLUS, MINUS, PLUS_DOT, MINUS_DOT)    |
| <code>_term()</code>     | Multiplication / division (STAR, SLASH, STAR_DOT, SLASH_DOT) |
| <code>_factor()</code>   | Literals, identifiers, calls, grouped expressions            |

#### Grammar Rules Applied

The productions below fire for each construct in the factorial program. **Uppercase names are terminal token types** (from the Token Grammar); lowercase names are non-terminal symbols.

```

(* Function definition *)
func_def -> DEF IDENTIFIER LPAREN params RPAREN block
params -> IDENTIFIER

```

```

block -> LBRACE stmt_list RBRACE

(* Statements inside the function body *)
assignment -> IDENTIFIER ASSIGN expr SEMICOLON
while_stmt -> WHILE LPAREN expr RPAREN block
return_stmt -> RETURN expr SEMICOLON

(* Expressions used in the factorial program *)
expr -> comparison | additive
comparison -> additive BANG_EQUAL additive (* i != 0 *)
additive -> term MINUS term (* i - 1 *)
term -> factor STAR factor (* result * i *)
factor -> INT_LITERAL (* 1, 5, 0 *)
        | IDENTIFIER (* i, n, result *)
        | IDENTIFIER LPAREN args RPAREN (* factorial(x), print(f) *)

(* Top-level statements *)
assignment -> IDENTIFIER ASSIGN expr SEMICOLON (* x = 5; f = factorial(x); *)

```

**Note on if/else.** Although this program contains no if statement, the grammar rule uses token names

— not quoted strings — for every terminal:

```

if_stmt -> IF LPAREN expr RPAREN block else_part
else_part -> ELSE block | epsilon

```

## AST Produced

Program

```

FunctionDef name="factorial" params=["n"]
  Block
    Assign name="result"
      Literal value=1 (Integer)
    Assign name="i"
      Identifier name="n"
    While
      BinaryOp op="!="
        Identifier name="i"
        Literal value=0 (Integer)
      Block (loop body)
        Assign name="result"
          BinaryOp op="*"
            Identifier name="result"
            Identifier name="i"
        Assign name="i"
          BinaryOp op="-"
            Identifier name="i"
            Literal value=1 (Integer)
      Return
        Identifier name="result"
    Assign name="x"
      Literal value=5 (Integer)
    Assign name="f"
      FunctionCall name="factorial"
        Identifier name="x"
    FunctionCall name="print"

```

```
Identifier name="f"
```

Listing 2: Abstract Syntax Tree for the factorial program.

## 4 Stage 3 — Type Checking

The type checker performs a single-pass, syntax-directed AST traversal. Types are inferred from literals, operators, and call sites — no annotations are required. The first call `factorial(x)` fixes the parameter type `n : Integer`, after which the function body is checked in a dedicated scope.

### Function Body Type Inference

| Statement                        | Inferred type | Action                                                              |
|----------------------------------|---------------|---------------------------------------------------------------------|
| <code>result = 1</code>          | Integer       | Record <code>result : Integer</code> in function scope.             |
| <code>i = n</code>               | Integer       | Look up <code>n: Integer</code> . Record <code>i : Integer</code> . |
| <code>while (i != 0)</code>      | Boolean       | Both operands are Integer; <code>!=</code> yields Boolean.          |
| <code>result = result * i</code> | Integer       | <code>Integer × Integer → Integer</code> .                          |
| <code>i = i - 1</code>           | Integer       | <code>Integer − Integer → Integer</code> .                          |
| <code>return result</code>       | Integer       | Sets function return type to Integer.                               |

### Final Symbol Table

| Scope           | Name                   | Kind     | Type                               |
|-----------------|------------------------|----------|------------------------------------|
| Global          | <code>factorial</code> | Function | params: [Integer], return: Integer |
| Global          | <code>x</code>         | Variable | Integer                            |
| Global          | <code>f</code>         | Variable | Integer                            |
| factorial scope | <code>n</code>         | Variable | Integer                            |
| factorial scope | <code>result</code>    | Variable | Integer                            |
| factorial scope | <code>i</code>         | Variable | Integer                            |



**Recursion note.** The type checker tracks active calls in `_active_calls`. A recursive call such as `return n * factorial(n - 1)` would be rejected with `TypeCheckError: Recursive function calls are not supported: factorial`. The iterative formulation avoids this entirely.

## 5 Stage 4 — Interpretation

The tree-walking interpreter executes the validated AST using a chain of `_Environment` objects for scoped variable storage. Each function call creates a new environment linked to the caller's.

After binding the parameter and initialising locals, the function environment is:

$$call\_env = \{ n \mapsto 5, result \mapsto 1, i \mapsto 5 \}$$

Table ?? traces the while loop iterations.

Table 2: While loop execution trace for `factorial(5)`.

| Iter. | i | Condition | result = result * i  | i = i - 1   |
|-------|---|-----------|----------------------|-------------|
| 1     | 5 | true      | $1 \times 5 = 5$     | $5 - 1 = 4$ |
| 2     | 4 | true      | $5 \times 4 = 20$    | $4 - 1 = 3$ |
| 3     | 3 | true      | $20 \times 3 = 60$   | $3 - 1 = 2$ |
| 4     | 2 | true      | $60 \times 2 = 120$  | $2 - 1 = 1$ |
| 5     | 1 | true      | $120 \times 1 = 120$ | $1 - 1 = 0$ |
| 6     | 0 | false     | <i>loop exits</i>    |             |

After the loop:

$$call\_env = \{ n \mapsto 5, result \mapsto 120, i \mapsto 0 \}$$

`return result` raises `_ReturnSignal(120)`, caught by `_call_function()`, which returns 120. The global environment becomes `{x ↦ 5, f ↦ 120}`, and `print(f)` outputs:

120 : Integer

## 6 Pipeline Summary

Table 3: End-to-end pipeline summary for `factorial(5)`.

| Stage               | Result                                                                                                                                                                                        |
|---------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Lexer</b>        | 54 tokens emitted (plus EOF), including DEF, WHILE, RETURN, BANG_EQUAL, STAR, MINUS, several INT_LITERAL tokens, and multiple IDENTIFIER tokens.                                              |
| <b>Parser</b>       | A Program AST with one top-level FunctionDef (factorial), two top-level Assign nodes (x, f), and one top-level FunctionCall (print).                                                          |
| <b>Type Checker</b> | All six symbols (factorial, x, f, n, result, i) inferred as Integer. No type errors.                                                                                                          |
| <b>Interpreter</b>  | <code>factorial(5)</code> computed iteratively over 5 loop iterations: $1 \times 5 \times 4 \times 3 \times 2 \times 1 = 120$ . Global environment ends as $\{x \mapsto 5, f \mapsto 120\}$ . |
| <b>Output</b>       | 120 : Integer                                                                                                                                                                                 |